

Detección Multicolor en Tiempo Real

Código fuente comentado para el seguimiento de objetos RGB e YUV en espacio de color LAB

Descripción del Script: Este programa realiza el procesamiento de visión artificial directamente en la cámara OpenMV. Captura fotogramas a resolución QVGA (320x240) y detecta bloque a bloque (blobs) la posición cartesiana de objetos rojos, verdes, azules y amarillos mediante el espacio cromático CIE LAB.

CÓDIGO FUENTE DOCUMENTADO

```
import sensor # Librería para controlar el hardware de la cámara OpenMV
import time   # Librería para manejar tiempos y retardos
# =====
# CONFIGURACIÓN INICIAL DEL SENSOR DE IMAGEN
# =====
sensor.reset() # Reinicia la cámara a sus ajustes por defecto para asegurar un estado limpio
sensor.set_pixformat(sensor.RGB565) # Configura el formato a RGB565 (16 bits de color,
necesario para detectar colores)
sensor.set_framesize(sensor.QVGA) # Define la resolución a 320x240 píxeles
sensor.skip_frames(time=2000) # Espera 2000 ms (2 segundos) para que la cámara ajuste la
exposición y el brillo automáticamente
# =====
# UMBRALES DE COLOR (ESPACIO CROMÁTICO LAB)
# Formato: (L_min, L_max, A_min, A_max, B_min, B_max)
# L = Luminosidad | A = Verde/Rojo | B = Azul/Amarillo
# =====
ROJO = (30, 100, 15, 127, 15, 127) # Rango LAB calibrado para detectar color rojo
VERDE = (20, 80, -80, -10, 10, 80) # Rango LAB calibrado para detectar color verde
AZUL = (20, 80, 0, 40, -100, -10) # Rango LAB calibrado para detectar color azul
AMARILLO = (50, 100, -20, 20, 40, 100) # Rango LAB calibrado para detectar color amarillo
# =====
# BUCLE PRINCIPAL DE PROCESAMIENTO
# =====
while True:
    img = sensor.snapshot() # Captura un fotograma actual de la cámara y lo guarda en la
variable 'img'
    # -----
    # EVALUACIÓN DEL COLOR ROJO
    # -----
    blobs = img.find_blobs([ROJO])
    # Busca áreas o manchas (blobs) que coincidan con el umbral ROJO
    if len(blobs) > 0: # Si la lista de detecciones no está vacía (encontró al
menos uno)
        blob = blobs[0] # Toma la primera coincidencia (normalmente la más
relevante/grande)
        print("COLOR: ROJO") # Muestra en la consola el nombre del color
        print("X:", blob.cx) # Imprime la coordenada del centro del objeto en el eje X
(0 a 320)
```

```

        print("Y:", blob.cy)          # Imprime la coordenada del centro del objeto en el eje Y
(0 a 240)
# -----
# EVALUACIÓN DEL COLOR VERDE
# -----
blobs = img.find_blobs([VERDE]) # Busca áreas que coincidan con el umbral VERDE
if len(blobs) > 0:              # Si detectó algún objeto verde
    blob = blobs[0]             # Toma la primera coincidencia
    print("COLOR: VERDE")       # Muestra en consola que halló color verde
    print("X:", blob.cx)        # Imprime la posición horizontal X del centro
    print("Y:", blob.cy)        # Imprime la posición vertical Y del centro
# -----
# EVALUACIÓN DEL COLOR AZUL
# -----
blobs = img.find_blobs([AZUL])  # Busca áreas que coincidan con el umbral AZUL
if len(blobs) > 0:              # Si detectó algún objeto azul
    blob = blobs[0]             # Toma la primera coincidencia
    print("COLOR: AZUL")       # Muestra en consola que halló color azul
    print("X:", blob.cx)        # Imprime la posición horizontal X del centro
    print("Y:", blob.cy)        # Imprime la posición vertical Y del centro
# -----
# EVALUACIÓN DEL COLOR AMARILLO
# -----
blobs = img.find_blobs([AMARILLO]) # Busca áreas que coincidan con el umbral AMARILLO
if len(blobs) > 0:              # Si detectó algún objeto amarillo
    blob = blobs[0]             # Toma la primera coincidencia
    print("COLOR: AMARILLO")   # Muestra en consola que halló color amarillo
    print("X:", blob.cx)        # Imprime la posición horizontal X del centro
    print("Y:", blob.cy)        # Imprime la posición vertical Y del centro

```